

TECHNICAL DESIGN DOCUMENT

# Auto Geofence Punch Background Architecture & Design

How hands-free attendance IN/OUT works when the app is killed — covering the background worker, scheduler, gate system, and 37-test verification suite.

# | Contents

|    |                               |    |
|----|-------------------------------|----|
| 01 | Executive Summary             | 03 |
| 02 | System Architecture           | 04 |
| 03 | Geofence Monitoring Lifecycle | 06 |
| 04 | Gate System & Decision Logic  | 08 |
| 05 | Testing Strategy & Coverage   | 10 |
| 06 | Appendix                      | 12 |

## 01 · EXECUTIVE SUMMARY

# Executive Summary

The auto geofence punch feature automates attendance **IN** and **OUT** punches using GPS proximity to configured office geofences, operating as an Android foreground service launched by Workmanager even when the app has been killed. It monitors GPS position against a set of office zones, detects entry and exit events, and calls the punch API automatically — requiring zero user interaction for daily attendance.

## Key Takeaways

- **Zero-touch attendance:** IN/OUT punches happen without any user interaction, triggered purely by GPS location relative to office geofences.
- **Survives app termination:** On Android, Workmanager re-launches the foreground service at shift start even when the app was killed. iOS uses BGTaskScheduler with best-effort guarantees.
- **Four-layer gate protection:** GPS accuracy filtering, cooldown debouncing, server-status reconciliation, and shift-hours validation prevent false or duplicate punches.
- **37 unit tests:** A dedicated test suite verifies the full lifecycle, all gate conditions, error handling, and edge cases including disabled feature, no network, and rapid IN/OUT transitions.

### QUESTIONS THIS DOCUMENT ANSWERS

How does the background worker detect geo-proximity and decide to punch IN or OUT? What happens when the app is killed — how does the service restart? What gates prevent false or duplicate punches under real-world GPS conditions?

## 02 · SYSTEM ARCHITECTURE

# System Architecture

The geofence auto-punch system is composed of three layers that each own a distinct concern: the **entrypoint** manages the Dart isolate lifecycle and GPS stream, the **worker** owns proximity decisions and gate checks, and the **scheduler** handles alarm registration and persistence.

## Layer Overview

| Layer             | File                                         | Responsibility                                                                             | Key Classes                                                                                     |
|-------------------|----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Entrypoint</b> | <code>geofence_bg_entrypoint.dart</code>     | Isolate lifecycle, GPS subscription, accuracy gate, location filtering, confidence scoring | <code>geofenceBgEntrypoint</code> , <code>LocationFilter</code> , <code>ConfidenceScorer</code> |
| <b>Worker</b>     | <code>geofence_background_worker.dart</code> | Proximity detection, entry/exit decision logic, all gate checks, punch API calls           | <code>GeofenceBackgroundWorker</code> , <code>ExitTrendAnalyzer</code>                          |
| <b>Scheduler</b>  | <code>geofence_scheduler.dart</code>         | Workmanager task registration, shift alarm scheduling, persistence of alarm state          | <code>GeofenceScheduler</code> , <code>Workmanager</code>                                       |

## Entrypoint Layer

Marked with `@pragma('vm:entry-point')` , the entrypoint function `geofenceBgEntrypoint` runs in a separate Dart isolate owned by Android's foreground service. It initializes Flutter bindings, instantiates the worker, calls `loadData()` to fetch offices and shifts, then subscribes to `Geolocator.getPositionStream` with high accuracy. Each incoming GPS position passes through an accuracy gate (30m minimum), the `LocationFilter` for jump and speed filtering, and the `ConfidenceScorer` before reaching the worker.

## Worker Layer

`GeofenceBackgroundWorker` receives filtered location fixes and runs the full decision tree. It caches offices and shifts in memory, manages a shift-window timer, and tracks pending exit state. The worker accepts an optional `Dio?` parameter for test injection, avoiding the need for global HTTP mocking.

## Scheduler Layer

`GeofenceScheduler` registers a one-time Workmanager task ( `geofence_shift_start` ) with a delay calculated from the current time to the next shift start. When the task fires, it checks whether the background service is already running; if not, it configures `FlutterBackgroundService` with the geofence endpoint and starts it. The scheduler persists the next alarm time and shift name in `SharedPreferences` for diagnostics.

Android vs iOS reliability. Workmanager on Android uses WorkManager API, which survives app kill and device reboot. On iOS it falls back to BGTaskScheduler with an approximately 15-minute execution window. The geofence feature is designed for Android as the primary target; iOS is best-effort.

# Geofence Monitoring Lifecycle

The monitoring lifecycle spans from app termination through shift-end stop, covering 10 distinct stages. Each stage is independently testable and verified by the 37-test suite.

## Full Lifecycle Stages

1. Workmanager fires. The OS-triggered alarm calls `GeofenceScheduler._callback` with task name `geofence_shift_start`. The initial delay was set during the previous punch-out (or at app init).
2. Background service configured. If `FlutterBackgroundService` is not already running, the callback calls `svc.configure()` with the geofence entrypoint (`geofenceBgEntrypoint`), foreground mode, location notification channel, and `AndroidForegroundType.location`.
3. Service starts. `svc.startService()` launches a new Dart isolate running the entrypoint function. A heartbeat timer (every 5 min) keeps the service alive on Android.
4. Data loaded. The entrypoint calls `geoWorker.loadData()`, which builds a Dio client using the stored access token, then fetches office list and shift definitions from the API. Results are cached in memory for the duration of the service.
5. GPS stream subscribed. The entrypoint opens a `Geolocator.getPositionStream()` subscription with high accuracy, zero distance filter. On iOS, `allowBackgroundLocationUpdates` and `pauseLocationUpdatesAutomatically: false` are set.
6. Accuracy gate applied. Raw positions with `accuracy > 30m` (`MIN_ACCURACY`) are discarded immediately. The remainder pass through `LocationFilter.process()` for jump detection and speed smoothing.
7. Confidence scored. `ConfidenceScorer.score()` computes a confidence value based on accuracy, tracking state, and jump score. Scores below `CONFIDENCE_THRESHOLD` suppress entry detection.
8. Entry or exit detected. The worker's `onLocationFix()` runs entry check (distance to office center  $\leq$  geofence radius) and exit check (distance  $>$  radius + GPS margin, with trend confirmation). Both pass through cooldown, server-status, and shift-hours gates before calling the punch API.
9. Punch called & next shift scheduled. On successful punch-IN, the worker updates local state and invokes `gf_punch` on the service instance. On punch-OUT, it additionally calls `GeofenceScheduler.scheduleNextShift()` to register the next day's alarm, then invokes `stop` to shut down the GPS stream and foreground service.
10. Service stopped. The `stop` handler cancels the position subscription, cancels the heartbeat timer, sets `running = false`, and calls `service.stopSelf()`. The isolate terminates until the next Workmanager alarm.

Overtime handling. When the shift end time passes but the user is still punched IN, the worker keeps monitoring until an OUT is detected. Only after OUT does it schedule the next shift and stop the service.

## Shift Timer Management

The worker computes three shift states via `_scheduleShiftWindow()` :

| State        | Condition                                   | Action                                                             |
|--------------|---------------------------------------------|--------------------------------------------------------------------|
| Before shift | <code>now &lt; todayStart</code>            | Schedule a <code>Timer</code> to activate at start; GPS ignored    |
| In shift     | <code>todayStart ≤ now &lt; todayEnd</code> | Monitoring active; GPS processed                                   |
| After shift  | <code>now ≥ todayEnd</code>                 | If OUT: schedule tomorrow's timer; else keep monitoring (overtime) |

## 04 · GATE SYSTEM

# Gate System & Decision Logic

Before calling the punch API, every potential IN or OUT event passes through a series of **seven decision gates**. Each gate checks one axis of validity; all gates must pass for a punch to proceed.

| Gate                | Check                                                                                                              | Purpose                                                                   |
|---------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1. GPS Accuracy     | <code>raw.accuracy ≤ 30m</code>                                                                                    | Reject GPS fixes with poor accuracy (e.g. indoor drift, weak signal)      |
| 2. Entry Confidence | <code>confidence ≥ CONFIDENCE_THRESHOLD</code>                                                                     | Only punch IN when the location filter is confident the device is stable  |
| 3. Exit Trend       | Successive outside fixes confirmed by <code>ExitTrendAnalyzer</code>                                               | Avoid spurious OUT from momentary GPS noise outside radius                |
| 4. Cooldown         | <code>same direction &amp; &lt; 5 min since last punch</code>                                                      | Prevent rapid IN/OUT flips from borderline GPS positions                  |
| 5. Server Status    | <code>/todayStatus</code> API: <code>isPunchedIn</code> , <code>isPunchedOut</code> , <code>hasNotPunchedIn</code> | Reconcile with server state before any punch                              |
| 6. Shift Hours      | Local: <code>now ≥ todayStart</code> ; Server: <code>currentShift name match</code>                                | Respect configured shift start/end times                                  |
| 7. Feature Toggle   | <code>geofence_auto_enabled</code> <code>SharedPreference</code>                                                   | Allow users to disable auto geofence without revoking location permission |

## Gate 1 — GPS Accuracy

Applied at the entrypoint level before the position reaches the worker. The threshold is **30 meters** ( `LocationFilter.MIN_ACCURACY` ). Fixes with accuracy above this value are silently dropped. This is the first line of defense against noisy GPS data common near building entrances or in urban canyons.

## Gate 2 — Entry Confidence

The `ConfidenceScorer` combines accuracy, speed, tracking state, and jump score into a single value. Entry detection only fires when this value meets `CONFIDENCE_THRESHOLD` . This prevents IN punches during brief moments of GPS wander near the boundary.



## Gate 3 — Exit Trend

Exits are not triggered on a single outside fix. Instead, the worker calculates a [GPS margin](#) (max of 10m and the current accuracy), then starts tracking if the position is beyond the radius plus margin. Subsequent positions feed into `ExitTrendAnalyzer`, which confirms an exit trend over multiple samples. If the device re-enters the zone, the trend resets.

## Gate 4 — Cooldown

After any punch, subsequent punches of the [same direction](#) are blocked for 5 minutes. IN after OUT has a separate 2-minute debounce (shorter because transitioning from OUT to IN is common during lunch). Opposite-direction punches (IN after OUT past 2 min, or OUT after IN past 5 min) are allowed.

## Gate 5 — Server Status

Before punching, the worker calls `/todayStatus`. If the server reports the user is already punched IN, an IN punch is skipped. If it reports already punched OUT or no IN today, an OUT punch is skipped. Network failures at this gate cause the worker to skip IN (conservative) but allow OUT (best-effort).

## Gate 6 — Shift Hours

Two sub-checks: a local check using the cached shift's `todayStart`, and a server-side check using the `currentShift` name from the status response. Both must agree that the current time is within the shift window. This prevents early IN punches and after-hours punches even if the server is unreachable for the server check.

## Gate 7 — Feature Toggle

The `geofence_auto_enabled` boolean in `SharedPreferences` serves as a master switch. When disabled, the worker returns early from `onLocationFix` without any processing. This allows users to opt out without revoking location permissions.

Gate ordering matters. Cheap checks (feature toggle, accuracy) run first. Expensive checks (server API calls) run last. This minimizes network requests when a local gate would reject the punch anyway.

# Testing Strategy & Coverage

The test suite at `test/features/punch/services/geofence_background_worker_test.dart` contains 37 tests organized into 8 categories, covering the full worker lifecycle from data loading through punch API calls to service stop.

## Test Infrastructure

Tests use a `_FakeServiceInstance` that records all `invoke` and `stopSelf` calls, and a `_MockInterceptor` that simulates API responses while tracking punch state across calls. The worker is constructed with a test `Dio` instance, avoiding global HTTP mocking.

## Test Categories

| Category        | Count | What It Verifies                                                                                                                                                                                  |
|-----------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Loading    | 3     | Offices and shifts fetched successfully; skip when no token available                                                                                                                             |
| Entry Detection | 5     | Punch IN when inside geofence radius; debounce IN after recent OUT; multiple office selection                                                                                                     |
| Exit Detection  | 5     | Exit trend tracking when outside radius + margin; trend confirmation triggers OUT; re-entry resets pending state                                                                                  |
| Cooldown        | 4     | Same-direction cooldown within 5 min; expiry after 5+ min; opposite-direction not blocked                                                                                                         |
| Server Status   | 4     | Already punched IN skips IN; already punched OUT skips OUT; no IN today skips OUT; unreachable server skips IN                                                                                    |
| Shift Hours     | 4     | IN before shift start skipped (local check); server shift name mismatch blocks IN; post-shift OUT allowed                                                                                         |
| Full Lifecycle  | 4     | IN followed by OUT completes full cycle; OUT triggers stop; Workmanager alarm scheduled; daily repeat cycle                                                                                       |
| Edge Cases      | 8     | Disabled feature toggle; no shifts configured; no offices configured; missing access token; low GPS accuracy; already punched in at server level; rapid IN/OUT flips; error during punch API call |

TOTAL TESTS

37

Across 8 categories

CATEGORIES

8

Data loading through edge cases

## LIFECYCLE

4

IN → OUT → stop → repeat

## PRE-EXISTING

20

Unchanged, all pass

## Key Scenarios

### Rapid IN/OUT Flip

If a user walks near the geofence boundary and briefly crosses it, the cooldown gate prevents a second IN within 5 minutes of the first. The exit-trend gate prevents OUT on a single outside fix.

### App Killed, Shift Start

Workmanager fires the alarm. The callback checks service state, configures the background service if not running, and starts it. The entrypoint loads data and begins GPS monitoring. This path is verified by the lifecycle test category.

### Overtime Monitoring

If the shift end passes while the user is still IN, the worker keeps monitoring. Only after the OUT punch does it schedule the next shift and stop. This prevents an unintended early OUT at shift end.

# Appendix

## A. File Index

| File                                                                           | Lines | Purpose                                                                           |
|--------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------|
| <code>lib/features/punch/services/geofence_background_worker.dart</code>       | 452   | Core worker: proximity detection, gate logic, punch API calls                     |
| <code>lib/features/punch/services/geofence_bg_entrypoint.dart</code>           | 112   | Background isolate entrypoint: lifecycle, GPS stream, accuracy gate               |
| <code>lib/features/punch/services/geofence_scheduler.dart</code>               | 152   | Workmanager alarm registration, shift scheduling, persistence                     |
| <code>test/features/punch/services/geofence_background_worker_test.dart</code> | 770   | 37 tests covering full worker lifecycle                                           |
| <code>pubspec.yaml</code>                                                      | —     | Dependency declarations; <code>workmanager</code> bumped to <code>^0.9.0+3</code> |

## B. Glossary

**Geofence** — A virtual perimeter around a physical office location, defined by a center (lat/lng) and radius in meters.

**Workmanager** — Flutter plugin wrapping Android's `WorkManager` API and iOS `BGTaskScheduler` for persistent background task scheduling.

**Foreground service** — An Android service that runs with a persistent notification, less likely to be killed by the OS than a background service.

**ExitTrendAnalyzer** — Internal component that tracks successive GPS positions outside a geofence and confirms a genuine exit trend before allowing an OUT punch.

**GPS margin** — An adaptive buffer added to the geofence radius (max of 10m and current GPS accuracy) to prevent spurious exit detection from GPS noise.

**Cooldown** — A 5-minute window after any punch during which punches of the same direction are suppressed. IN after OUT has a shorter 2-minute debounce.

## C. References

- `pub.dev/packages/workmanager` — Background task scheduling API
- `pub.dev/packages/flutter_background_service` — Foreground service management
- `pub.dev/packages/geolocator` — GPS position streaming and distance calculation
- `geofence_background_worker.dart` — Core worker implementation (452 lines)
- `geofence_background_worker_test.dart` — 37-test verification suite (770 lines)